

Weekly Python Assignment 5 - Group 3 - S23

Instructions: For each exercise below, type the requested code into the cell below the exercise, then run your code to verify that it works correctly.

Color Code (nonfunctional in Google Colab):

- External File/Imported Module
- User-created Function
- Python/Imported Function
- Name-specific Data Structure/Variable

Relative Difficulty & Complexity

- **Easy (E):** Covered during the week's lecture. Typically single-step exercise
- **Medium (M):** Some combination with topics previously covered. Mostly double-step exercise
- **Hard (H):** Combination with topics previously covered. Multi-step exercise

Breakdown: 3 (E), 5 (M), & 2 (H)

1(E). Print the sentence "I would rather be doing almost anything else than this homework." in reverse.

```
In [2]: sentence = "I would rather be doing almost anything else than this homework."
for i in range (len(sentence)-1, -1, -1):
    print(sentence[i], end = "")
# or make the sentence in reverse
print("")
reverse_sentence = sentence[::-1]
print(reverse_sentence)
```

```
.krowemoh siht naht esle gnihtyna tsomla gniod eb rehtar dluow I
.krowemoh siht naht esle gnihtyna tsomla gniod eb rehtar dluow I
```

2(M). Create a list of characters from the sentence "I would rather be doing almost anything else than this homework.", including spaces and symbols. Print out the entire list as is. After that, join the list of characters back together again to recreate the sentence and save it to a different variable name. Then print out the recreated string.

```
In [11]: char_lst = []
char_lst = list(sentence)
print(char_lst)

#List to string by using join()
created_sentence = ''.join(char_lst)
print(created_sentence)
```

```
['I', ' ', 'w', 'o', 'u', 'l', 'd', ' ', 'r', 'a', 't', 'h', 'e', 'r', ' ', 'b', 'e', ' ', ' ', 'd', 'o', 'i', 'n', 'g', ' ', 'a', 'l', 'm', 'o', 's', 't', ' ', ' ', 'a', 'n', 'y', 't', 'h', 'i', 'n', 'g', ' ', ' ', 'e', 'l', 's', 'e', ' ', ' ', 't', 'h', 'a', 'n', ' ', ' ', 't', 'h', 'i', 's', ' ', ' ', 'h', 'o', 'm', 'e', 'w', 'o', 'r', 'k', '.']
```

I would rather be doing almost anything else than this homework.

3(H). Find the index of the letter "a" at the beginning of the word "almost" inside the sentence "I would rather be doing almost anything else than this homework."

Hint: You can use the list of characters created in Ex. 02 to start.

```
In [43]: # This function can find the other words, too.
def find_words_idx(sentence, word):
    check_sentence = " " + sentence
    for i in range(1, len(check_sentence)):
        if check_sentence[i-1] == " " and check_sentence[i] == word[0]: #if the first
            temp = check_sentence[i:i+len(word)] # check the next 5 letters after the
            if temp == word: # if find return index
                return (i-1) + word.index('a') # exact index of the 'a' in the word.

sentence = "I would rather be doing almost anything else than this homework."
print(find_words_idx(sentence, "almost"))
```

24

4(M). Replace the words "almost anything" with "absolutely nothing" in the sentence "I would rather be doing almost anything else than this homework." then print out the revised sentence.

```
In [47]: print(sentence)
new_sentence = sentence.replace("almost anything", "absolutely nothing")
print(new_sentence)
```

I would rather be doing almost anything else than this homework.

I would rather be doing absolutely nothing else than this homework.

5(H). Import any necessary module, if it has not been imported. Write a function that simulates a roll of dice in the game of **Risk** with the basis of the attacker can roll up to 3 dice, the defender can roll one or two. The highest roll of the attacker's dice is compared to the highest role of the defender's dice such that the outcome is decided as follows:

- If that attacker's die is greater than that defenders die, the defender loses one army.
- If the defender's die is greater than or equal to that attacker's die, the attacker loses an army.
- If the attacker has rolled two or three dice, and the defender has rolled two, the second highest attacker's die is then compared to the second highest defender's die, with the same rules as for the first die.

The function will accept two parameters: the first parameter is the number of attacker dice (1, 2, or 3) and the second parameter is the number of defender dice (1 or 2). The function will return the outcome of the battle (2 attacker armies lost, 1 attacker and 1 defender army lost, two defender armies lost, 1 defender army lost, or 1 attacker army lost). Run your function several times with a variety of inputs.

In [149]...

```

import random
def game(attacker, defender):
    attacker_die = sorted([random.randint(1, 6) for _ in range (attacker)])
    defender_die = sorted([random.randint(1, 6) for _ in range (defender)])
    attacker_lose = 0
    defender_lose = 0
    # first_game
    if attacker_die[-1] <= defender_die[-1]:
        attacker_lose += 1
    else:
        defender_lose += 1

    # second_game
    if attacker >= 2 and defender >= 2:
        if attacker_die[-2] <= defender_die[-2]:
            attacker_lose += 1
        else:
            defender_lose += 1

    return (attacker_lose, defender_lose)

a, b = game(3, 2)
print("%d attacker arimes lost, %d defender arimes lost" % (a, b))
a, b = game(3, 1)
print("%d attacker arimes lost, %d defender arimes lost" % (a, b))
a, b = game(1, 1)
print("%d attacker arimes lost, %d defender arimes lost" % (a, b))
a, b = game(2, 2)
print("%d attacker arimes lost, %d defender arimes lost" % (a, b))

```

```

0 attacker arimes lost, 2 defender arimes lost
0 attacker arimes lost, 1 defender arimes lost
1 attacker arimes lost, 0 defender arimes lost
1 attacker arimes lost, 1 defender arimes lost

```

6(M). Import `matplotlib.pyplot` module and give it the abbreviation `plt`. Also, import `math`, if it has not been imported. Plot the function:

$$y = \sinh(x)e^x$$

from $x=2$ to $x=\pi$.

Remember to include details for an informative plot (such as legends, titles, axis labels,...).

In [35]:

```

import matplotlib.pyplot as plt
import math

def y(x):
    return math.sinh(x) * math.exp(x)
points = 1000
x_values = [] # it will be 2 - 1000.. without using numpy..
starting = 2
for i in range(1, points + 2):
    x_values.append(starting)
    starting = 2 + ((math.pi - 2) / 1000 * i)

```

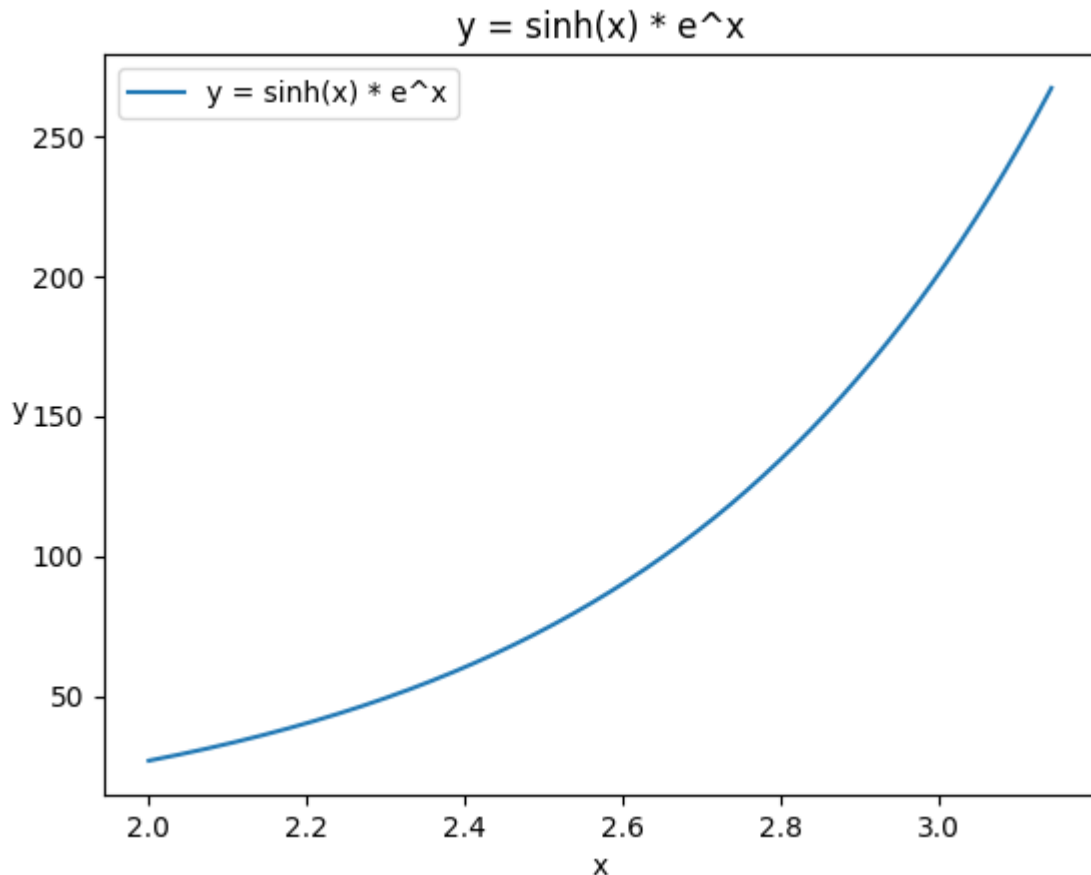
```

y_values = [y(x) for x in x_values]

plt.plot(x_values, y_values, label = "y = sinh(x) * e^x")
plt.title("y = sinh(x) * e^x")
plt.xlabel("x")
plt.ylabel('y', rotation = 0)

plt.legend()
plt.show()

```



7(E). Create a data structure called `weights` with each of its elements being a list of two values: the first value is a string with the date information, and the second value is the list of data.

Populate `weights` with the following data:

December 15-21, 2019: [151.6,152,151.8,151.6,150.8,152.2,151.2]

December 22-28, 2019: [153.2,152.4,150.8,151.6,154.4,153.8,152.2]

December 29, 2019 - January 4, 2020 [152.8,152.4,153,154.8,153.8,151.2,152]

January 5-11, 2020 [150.8,150.2,149.8,149.8,150.4,150.6,1]

January 12-18, 2020 [151,150,150.2,152.2,152.4,152.4,153.2]

January 19-25, 2020 [153,152.4,151.2,151.2,151,151.2,151.2]

January 26-February 3, 2020 [152,150.8,151.2,150,150.8,152.4,151.2]

Print out `weights` as is.

```

In [38]: weights = [
    ["December 15-21, 2019:", [151.6, 152, 151.8, 151.6, 150.8, 152.2, 151.2]],

```

```

["December 22-28, 2019:", [153.2, 152.4, 150.8, 151.6, 154.4, 153.8, 152.2]],
["December 29, 2019 - January 4, 2020:", [152.8, 152.4, 153, 154.8, 153.8, 151.2,
["January 5-11, 2020:", [150.8, 150.2, 149.8, 149.8, 150.4, 150.6, 1]],
["January 12-18, 2020:", [151, 150, 150.2, 152.2, 152.4, 152.4, 153.2]],
["January 19-25, 2020:", [153, 152.4, 151.2, 151.2, 151, 151.2, 151.2]],
["January 26-February 3, 2020:", [152, 150.8, 151.2, 150, 150.8, 152.4, 151.2]],
]

for i in weights:
    print(i)

```

```

['December 15-21, 2019:', [151.6, 152, 151.8, 151.6, 150.8, 152.2, 151.2]]
['December 22-28, 2019:', [153.2, 152.4, 150.8, 151.6, 154.4, 153.8, 152.2]]
['December 29, 2019 - January 4, 2020:', [152.8, 152.4, 153, 154.8, 153.8, 151.2, 15
2]]
['January 5-11, 2020:', [150.8, 150.2, 149.8, 149.8, 150.4, 150.6, 1]]
['January 12-18, 2020:', [151, 150, 150.2, 152.2, 152.4, 152.4, 153.2]]
['January 19-25, 2020:', [153, 152.4, 151.2, 151.2, 151, 151.2, 151.2]]
['January 26-February 3, 2020:', [152, 150.8, 151.2, 150, 150.8, 152.4, 151.2]]

```

8(M). Import any necessary module, if it has not been imported. Using the data structure `weights` created in Ex. 07, calculate the average weight for each week, then determine and print out the highest and lowest average weight for the seven weeks. Plot all seven averages so you can visually see how they compare.

Remember to include details for an informative plot (such as legends, titles, axis labels,...).

```

In [56]: average = []
week = []
for i in weights:
    temp = (sum(i[1]) / len(i[1]))
    week.append(i[0])
    average.append(temp)
min_avg = 1000
max_avg = -1

for ii in range(len(week)):
    min_avg = min(min_avg, average[ii])
    max_avg = max(max_avg, average[ii])
    print(week[ii], end = " ")
    print(average[ii])
print("-----")
print("min_avg and max_avg")
print(week[(average.index(min_avg))], min_avg)
print(week[(average.index(max_avg))], max_avg)
print("-----")

plt.plot(week, average, label = "Average weight")
plt.title("Average weight for each week")
plt.xlabel("week")
plt.ylabel('average', rotation = 0)

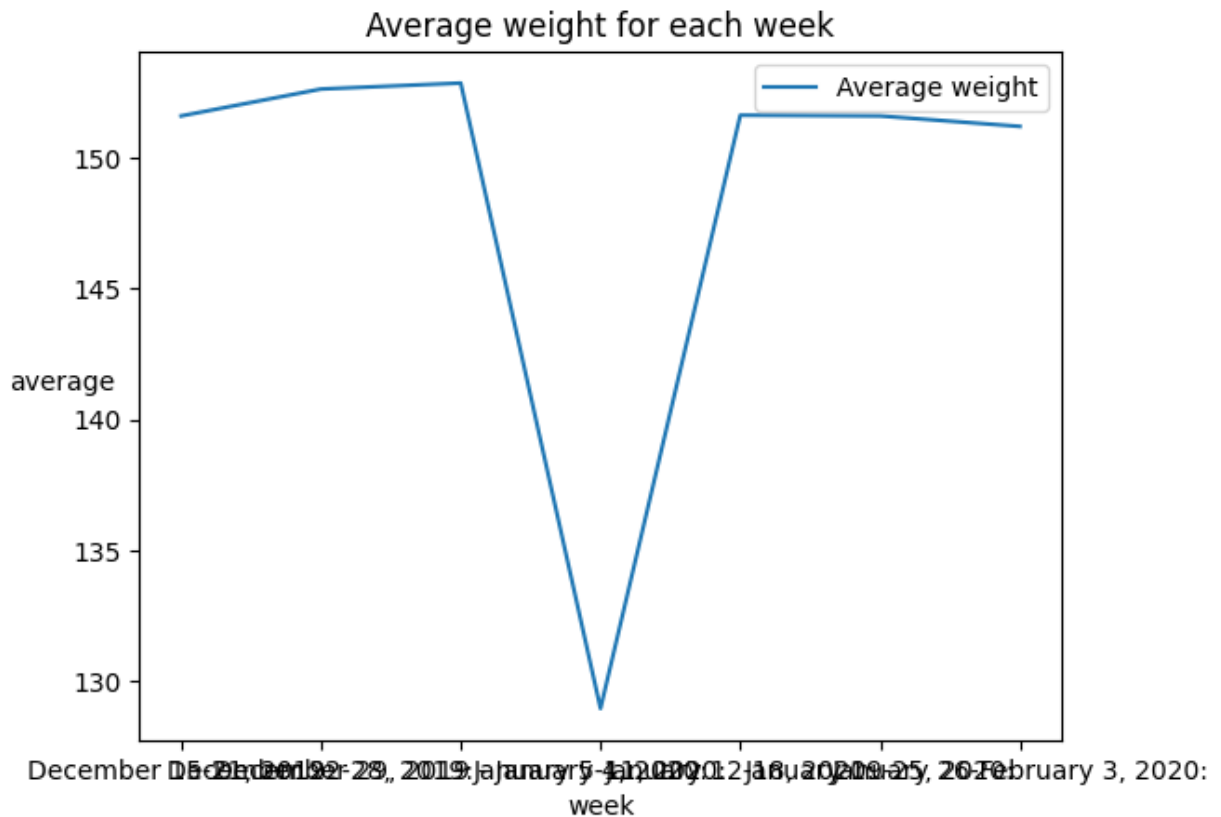
plt.legend()
plt.show()

```

December 15-21, 2019: 151.6
 December 22-28, 2019: 152.62857142857143
 December 29, 2019 - January 4, 2020: 152.85714285714286
 January 5-11, 2020: 128.94285714285715
 January 12-18, 2020: 151.6285714285714
 January 19-25, 2020: 151.6
 January 26-February 3, 2020: 151.2

 min_avg and max_avg

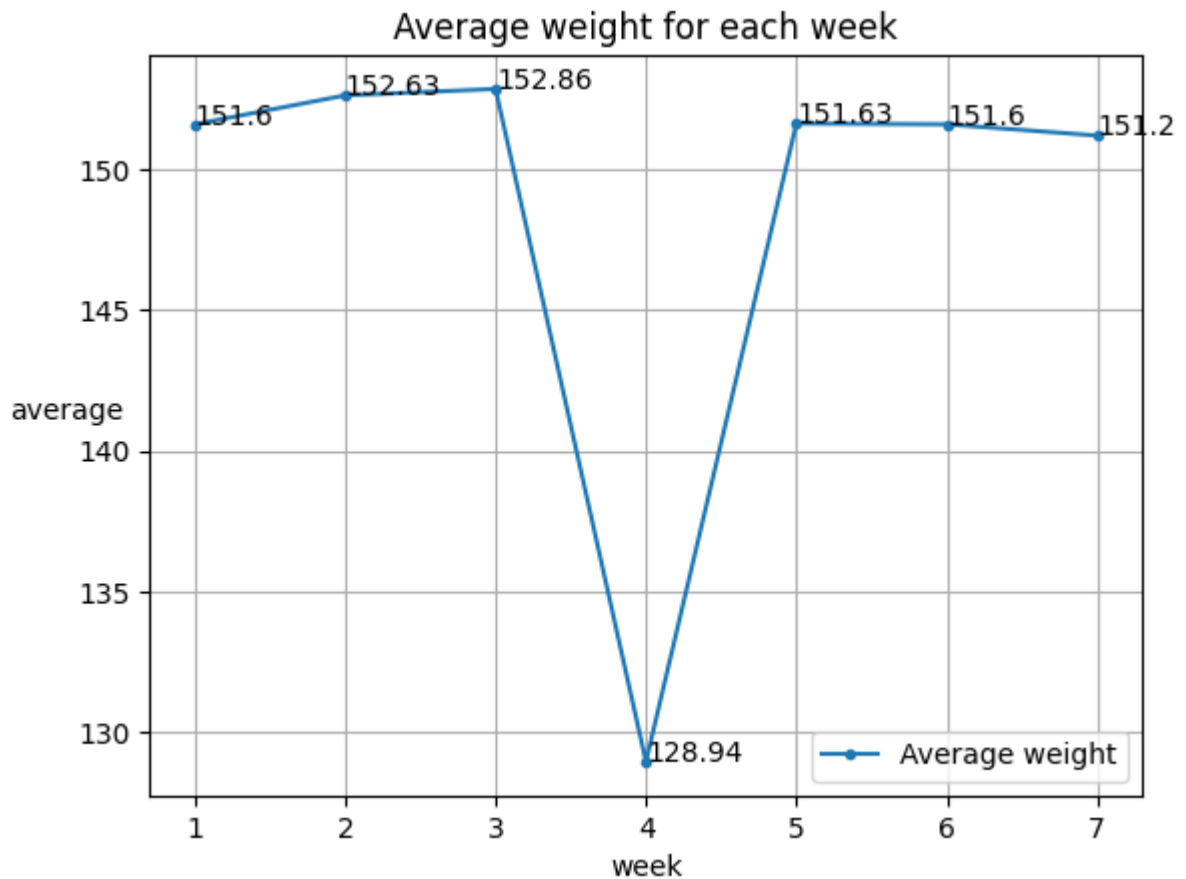
January 5-11, 2020: 128.94285714285715
 December 29, 2019 - January 4, 2020: 152.85714285714286



```
In [71]: # the information of week is too long,
week2 = [1, 2, 3, 4, 5, 6, 7]
plt.plot(week2, average, marker='o', markersize=3, label = "Average weight" )
plt.title("Average weight for each week")
for index in range(len(week2)):
    plt.text(week2[index], average[index], round(average[index], 2), fontsize=10)

plt.xlabel("week")
plt.ylabel('average', rotation = 0)
plt.grid()
plt.legend(loc = "lower right")
plt.show()

# So we can see that 3week average is highest weight and 4 week average is lowest weight
# This is because in the 4th week's data has the error value that is 1, So we can see
```

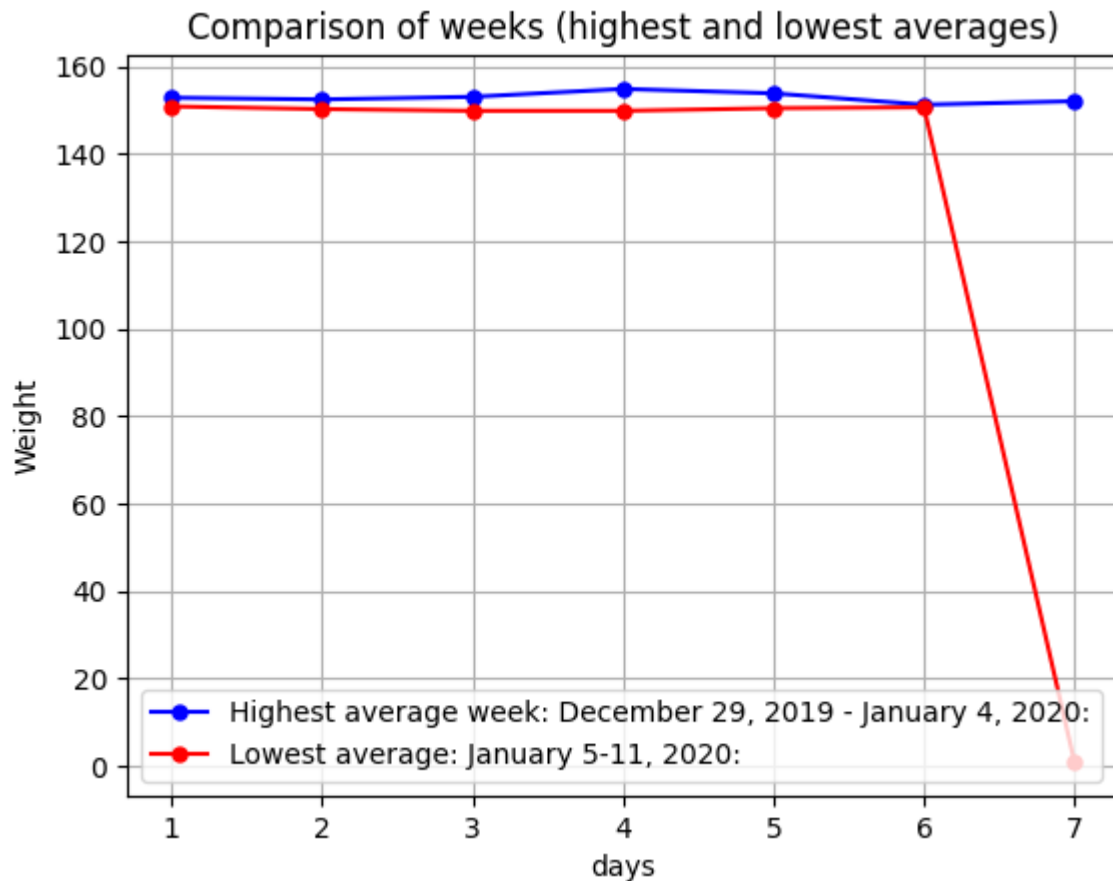


9(E). Import any necessary module, if it has not been imported. Using the data structure `weights` created in Ex. 07, Plot all the data from the weeks having the highest and lowest averages against their indices on a single chart so you can visually compare the two weeks. Be sure to use different colors and symbols for each week.

Remember to include details for an informative plot (such as legends, titles, axis labels,...).

```
In [83]: max_index = average.index(max(average))
min_index = average.index(min(average))

days = [i for i in range(1, 8)]
plt.plot(days, weights[max_index][1], 'bo-', markersize = 5, label = f"Highest average")
plt.plot(days, weights[min_index][1], 'ro-', markersize=5, label=f"Lowest average: {weights[min_index][1]}")
plt.title("Comparison of weeks (highest and lowest averages)")
plt.xlabel("days")
plt.ylabel("Weight")
plt.legend(loc = "lower left")
plt.grid()
plt.show()
```



10(M). Import any necessary module, if it has not been imported. Write the data structure `weights` to `weights_output.csv` such that each week and its data should be a row in the file. Once you have written the file, create a new data structure called `weights2` then read the contents of `weights_output.csv` into `weights2`. Print each element of `weights` and `weights2` and confirm that the data is preserved correctly.

```
In [110... # Process writing
import csv
weights = [
    ["December 15-21, 2019:", [151.6, 152, 151.8, 151.6, 150.8, 152.2, 151.2]],
    ["December 22-28, 2019:", [153.2, 152.4, 150.8, 151.6, 154.4, 153.8, 152.2]],
    ["December 29, 2019 - January 4, 2020:", [152.8, 152.4, 153, 154.8, 153.8, 151.2,
    ["January 5-11, 2020:", [150.8, 150.2, 149.8, 149.8, 150.4, 150.6, 1]],
    ["January 12-18, 2020:", [151, 150, 150.2, 152.2, 152.4, 152.4, 153.2]],
    ["January 19-25, 2020:", [153, 152.4, 151.2, 151.2, 151, 151.2, 151.2]],
    ["January 26-February 3, 2020:", [152, 150.8, 151.2, 150, 150.8, 152.4, 151.2]],
]

week_header = []
day_data = []
for i in weights:
    week_header.append(i[0])
    day_data.append(i[1])

with open('weights_output.csv', 'w', newline='') as csvfile:
    writer = csv.writer(csvfile)
    for i in range(7):
```

```
row = [week_header[i]] + day_data[i]
writer.writerow(row)
```

```
In [118... # Process read
weights2 = []
with open("weights_output.csv", "r") as csvfile:
    reader = csv.reader(csvfile)
    for row in reader:
        weights2.append([row[0], list(map(float, row[1:]))]) # the value in the csv fi
```

```
In [119... # compare weights and weights2

for i in weights:
    print(i)
print("-----")
for i in weights2:
    print(i)

['December 15-21, 2019:', [151.6, 152, 151.8, 151.6, 150.8, 152.2, 151.2]]
['December 22-28, 2019:', [153.2, 152.4, 150.8, 151.6, 154.4, 153.8, 152.2]]
['December 29, 2019 - January 4, 2020:', [152.8, 152.4, 153, 154.8, 153.8, 151.2, 15
2]]
['January 5-11, 2020:', [150.8, 150.2, 149.8, 149.8, 150.4, 150.6, 1]]
['January 12-18, 2020:', [151, 150, 150.2, 152.2, 152.4, 152.4, 153.2]]
['January 19-25, 2020:', [153, 152.4, 151.2, 151.2, 151, 151.2, 151.2]]
['January 26-February 3, 2020:', [152, 150.8, 151.2, 150, 150.8, 152.4, 151.2]]
-----
['December 15-21, 2019:', [151.6, 152.0, 151.8, 151.6, 150.8, 152.2, 151.2]]
['December 22-28, 2019:', [153.2, 152.4, 150.8, 151.6, 154.4, 153.8, 152.2]]
['December 29, 2019 - January 4, 2020:', [152.8, 152.4, 153.0, 154.8, 153.8, 151.2, 1
52.0]]
['January 5-11, 2020:', [150.8, 150.2, 149.8, 149.8, 150.4, 150.6, 1.0]]
['January 12-18, 2020:', [151.0, 150.0, 150.2, 152.2, 152.4, 152.4, 153.2]]
['January 19-25, 2020:', [153.0, 152.4, 151.2, 151.2, 151.0, 151.2, 151.2]]
['January 26-February 3, 2020:', [152.0, 150.8, 151.2, 150.0, 150.8, 152.4, 151.2]]
```